Table des matières

Cours 09 — Les codes 1

Cours 09 — Les codes

Cours 9/12 · La Brûlerie — Formation Data Engineering

Sources	**Ingestion**	**RAW**
d'où vient la donnée	aller la chercher, et la charger	la déposer, la ranger, la traiter
	✓ ici	

À ouvrir après avoir essayé. Le code garde les commentaires numérotés du cours, pour comparer ligne par ligne. Les requêtes se tapent dans le notebook de DuckDB (`duckdb -ui`, lancé depuis `Projet_Brulerie`), une par cellule ; le script se crée dans la première fenêtre, par le Bloc-notes.

1.2 Les lots du mois au-dessus de chaque seuil

```
-- 1. la plus haute température du grain de chaque lot
WITH lots AS (
    SELECT lot_id, MAX(CAST(temp_grain AS DECIMAL(5, 1))) AS grain_max
    FROM entrepot.raw_courbes
    GROUP BY lot_id
)
-- 2. pour chaque seuil, les lots qui l'ont dépassé
SELECT seuil, COUNT(*) FILTER (WHERE grain_max > seuil) AS lots
FROM lots CROSS JOIN range(215, 221) AS r(seuil)
GROUP BY seuil
ORDER BY seuil;
```

01_de/consommer.py

```
# 01_de/consommer.py : lire le journal du direct depuis son décalage, sans doublon, et alerter
import json
import sys
from pathlib import Path
import duckdb

JOURNAL, DECALAGE = Path("plateforme/direct/journal.jsonl"), Path("plateforme/direct/decalage.txt")
```


© 2026 iAlexMG. Tous droits réservés.

```
def consommer(con, seuil):
    con.execute("CREATE TABLE IF NOT EXISTS direct_courbes (lot_id TEXT, seconde INTEGER, instant TIMESTAMP, "
                "temp_grain DECIMAL(5, 1), temp_air DECIMAL(5, 1), evenement TEXT, PRIMARY KEY (lot_id, seconde))")
    # 1. reprendre au décalage : le rang du premier message pas encore lu
    decalage = int(DECALAGE.read_text()) if DECALAGE.exists() else 0
    messages = [json.loads(l) for l in JOURNAL.read_text(encoding="utf-8").splitlines()[decalage:]]
    nouveaux = 0
    for m in messages:
        # 2. une lecture déjà écrite : la clé (lot, seconde) la refuse, et l'écriture rend 0
        ecrit = con.execute("INSERT INTO direct_courbes VALUES ($lot_id, $seconde, $instant, $temp_grain, "
                            "$temp_air, $evenement) ON CONFLICT DO NOTHING", m).fetchone()[0]
        nouveaux += ecrit
        # 3. l'alerte, datée de l'heure de l'événement, que le message porte
        if ecrit and m["temp_grain"] > seuil:
            print(f"ALERTE {m['instant']} : lot {m['lot_id']}, grain à {m['temp_grain']} °C")
    # 4. le décalage, enregistré après l'écriture : une panne entre les deux fait relire, sans doubler
    DECALAGE.write_text(str(decalage + len(messages)))
    print(f"{len(messages)} messages lus depuis le décalage {decalage}, {nouveaux} lectures nouvelles")

if __name__ == "__main__":
    with duckdb.connect("plateforme/entrepot/brulerie.duckdb") as con:
        consommer(con, float(sys.argv[1]))
```

7.1 Le direct contre le fichier du jour, dans les deux sens

```
-- 1. les lectures du 10 juin, telles que le fichier du jour les a livrées au cold path
WITH fichier AS (
    SELECT lot_id, CAST(seconde AS INTEGER) AS seconde, CAST(temp_grain AS DECIMAL(5, 1)) AS temp_grain,
           CAST(temp_air AS DECIMAL(5, 1)) AS temp_air, evenement
    FROM entrepot.raw_courbes
    WHERE CONTAINS(filename, '2025-06-10')
),
direct AS (SELECT lot_id, seconde, temp_grain, temp_air, evenement FROM entrepot.direct_courbes)
-- 2. dans les deux sens : ce qui manque au direct, puis ce qu'il a en trop
SELECT (SELECT COUNT(*) FROM direct) AS lectures,
       (SELECT COUNT(*) FROM (SELECT * FROM fichier EXCEPT ALL SELECT * FROM direct)) AS manquent,
       (SELECT COUNT(*) FROM (SELECT * FROM direct EXCEPT ALL SELECT * FROM fichier)) AS en_trop;
```


© 2026 iAlexMG. Tous droits réservés.